1. Minimal pipeline (MVP you can demo)

Goal: given 2–4 drum & bass tracks (no vocals assumption), make an automatic transition that's beat-aligned and sounds DJ-ish.

MVP steps:

1. **Load audio** (wav/mp3) → `librosa.load(...)` or `soundfile`.
2. **Beat tracking** on each track → get beat times.
3. **(Optional-light) phrase-ish points** = just pick every 16 or 32 beats as a "section" instead of doing full self-similarity/Foote.
4. **Pick a transition point** in Track 1 (e.g. beat 96) and an intro point in Track 2 (e.g. its first strong beat).
5. **Tempo match**: time-stretch Track 2 to Track 1's tempo (`librosa.effects.time_stretch`).
6. **Crossfade** over N beats (e.g. 16 beats) with a simple curve.
7. **Export** mixed audio.

That alone shows: "we analyzed beats, aligned them, tempo-matched, and blended."

That's the core of the paper's "DJ tasks" without doing key detection, theme vectors, vocal avoidance, or learned downbeat detection.

---

2. What to cut (staying honest to the paper)

Here are concrete dials you can turn down:

1. **Assume DnB only + similar tempo**
   Paper already does genre-specific. You can go further: only tracks 170–176 BPM → makes tempo estimation easier and stretching lighter.
2. **No vocals**
   You already said this—so skip vocal-activity detection and "avoid vocal overlap" logic.
3. **No harmonic mixing (key detection)**
   That's a whole subproblem. Say: "for this prototype we assume key-compatible tracks / pre-curated playlist."
4. **Fake structural segmentation**
   Instead of computing a self-similarity matrix + Foote, do:
   - detect beats,
   - group into bars = 4 beats,
   - define "sections" = every 8 bars (32 beats).
     That enforces the "mix on musical boundaries" idea without doing the heavy part.

5. **Single transition type**
   The paper has double drop / rolling / relaxed. You can implement **just 'rolling'** (fade out A main → fade in B main) and keep lengths fixed (e.g. 16 bars).
6. **No auto track selection**
   Let the user provide an ordered playlist (`playlist = ["a.wav", "b.wav", "c.wav"]`). You can still write the function signature for "choose next track" to show you know where it would go.
7. **Precompute / skip ML downbeat classifier**
   Instead of classifying 1–2–3–4, assume bar starts at the first beat you get from `librosa.beat.beat_track` and keep counting in 4s. For DnB this will look okay enough for class demo.

That's all totally defensible in a writeup: "we focus on real-time-feasible, genre-constrained automatic transitions."

---

3. Partner split

Think of it as **Analysis Person** vs **Synthesis/Mixing Person**. You can swap, but this is clean.

Partner A – "Analysis & Markers"

Focus: all the "figure out where to mix" logic.

- `audio_io.py`: load audio, resample, mono.
- `beat_tracking.py`:
  - wrap `librosa.beat.beat_track(y, sr)` → return bpm + beat_times.
  - helper to group beats into bars (every 4 beats).
- `structure.py` (lightweight):
  - given beat_times, make "section" markers every 16 or 32 beats.
  - expose: `find_good_mix_point(track1_beats)` → returns time in seconds.
- (optional) small plotting to verify beats land on kicks.

Deliverable for A: given a filename, return a dict:

```
{
  "y": np.ndarray,
  "sr": 44100,
  "bpm": 174.0,
  "beat_times": np.array([...]),
  "section_times": np.array([...])
```

```
}
```

This makes Partner B's life easy.

Partner B – "Time-stretch & Crossfade Engine"

Focus: "I have two analyzed tracks; make them one."

- `time_align.py`:
  - function to match BPM: `stretch_to_bpm(y, sr, src_bpm, target_bpm)`
- `transition.py`:
  - function `make_transition(trackA, trackB, a_mix_time, b_start_beat, n_beats=16)`:
    1. cut A around `a_mix_time`
    2. align B so its chosen beat = `a_mix_time`
    3. apply crossfade envelope (linear or equal-power)
    4. sum
- `render.py`:
  - stitch: A_before + transition + B_after
  - write to wav using `soundfile.write`

Deliverable for B: a function

```
mix_two_tracks(trackA_meta, trackB_meta, output_path)
```

that calls Partner A's analysis to figure out the markers.

---

4. Day-by-day plan (it's Tuesday → finish by Friday/Saturday)

Today (Tue): Scoping + skeletons

- Agree on simplifications (the 7 above).

Make repo with folders:
`autodj/`

```
  analysis/  (A works here)
  mixer/     (B works here)
  main.py
data/        (songs)
```

-
- Both install `librosa`, `soundfile`, `numpy`.

Wed: Analysis working on 1 song

- Partner A:
    - `load_audio(path) -> y, sr`
    - `analyze_track(path) -> bpm, beat_times, section_times`
    - test on 1 DnB track and print first 20 beat times
- Partner B:

write crossfade helper:

```
def crossfade(a, b, fade_len):
    # a, b are same length arrays
```

    -
    - write tempo-stretch helper using librosa

Thu: Join

- Partner A:
    - guarantee beat_times as seconds and as sample indices
    - choose a simple rule for mix point: “4 bars before the end” → needs track duration
- Partner B:
    - given two analyzed JSONs, align and make a transition buffer
    - output a WAV

Fri: Polish + demo

- Make `main.py` that:
    1. reads a list of files
    2. analyzes all
    3. mixes 1→2→3 with fixed rules
- Add comments + short writeup on what you didn’t implement.

If you stick to that, by Friday you can hit “run” and get a mix file.

---

5. Rough code layout

```
autodj/
  main.py
  analysis/
    __init__.py
    audio_io.py
    beats.py
    structure.py
  mixer/
    __init__.py
    timestretch.py
    transition.py
    render.py
```

`analysis/beats.py` (Partner A)

```
import librosa
import numpy as np

def analyze_track(path):
    y, sr = librosa.load(path, sr=None, mono=True)
    tempo, beat_frames = librosa.beat.beat_track(y=y, sr=sr)
    beat_times = librosa.frames_to_time(beat_frames, sr=sr)
    # simple “structure”: every 32 beats is a section
    section_times = beat_times[::32]
    return {
        "path": path,
        "y": y,
        "sr": sr,
        "bpm": float(tempo),
        "beat_times": beat_times,
        "section_times": section_times,
    }
```

`mixer/timestretch.py` (Partner B)

```
import librosa
```

```
def stretch_to_bpm(y, orig_bpm, target_bpm):
    rate = target_bpm / orig_bpm
    return librosa.effects.time_stretch(y, rate)
```

Then transition just lines up on a beat.

Extra scope-limit ideas (if prof wants more “DSP”)

Since it’s an **embedded DSP lab**, they might like if you say:

- “We fixed the transition length in **beats** not seconds → musically consistent.”
- “We used EQ slots but implemented them as simple 2nd-order IIR low/high shelves on each track during overlap.”
- “We pre-stretched before mix to avoid runtime load.”

You can even say: “future embedded port would swap `librosa`’s phase vocoder for an STFT-based OLA we implement.”

If you want, I can turn this into a short README you can drop in the repo so both of you follow the same spec.